Name: **Shrey Pendurkar**
Class: **D15C**
Batch: **B**
Roll No: **38**

MLDL – Experiment 3

Aim

Apply Decision Tree and Random Forest for classification tasks

---

Dataset Source

- Dataset: Breast Cancer Wisconsin (Diagnostic) Dataset
- Repository: UCI Machine Learning Repository
- Access: sklearn.datasets.load_breast_cancer()
- URL: https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic)

Dataset Description

The Breast Cancer Wisconsin (Diagnostic) dataset is a widely used benchmark for binary classification. Features are computed from digitized images of fine needle aspirate (FNA) of breast tissue and describe the characteristics of the cell nuclei present. The task is to classify a tumour as either malignant (cancerous) or benign (non-cancerous).

- Total Instances: 569 samples (no missing values)
- Features: 30 real-valued numeric features
- Target Classes: Malignant (0) — 212 samples | Benign (1) — 357 samples
- Feature Groups: 10 core measurements (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension), each computed as mean, standard error, and worst value — yielding 30 features in total
- Origin: Digitized FNA images from the University of Wisconsin Hospitals

---

1. Decision Tree Classifier

Mathematical Formulation of the Algorithm

A Decision Tree recursively partitions the feature space by selecting the split (feature + threshold) that maximises the purity of the resulting child nodes. Purity is quantified using one of the following criteria.

Gini Impurity (default criterion):

$$Gini(t) = 1 - \Sigma_i \; p(i|t)^2$$

where p(i|t) is the proportion of class i samples at node t.

Entropy:

$$Entropy(t) = -\Sigma_i \; p(i|t) \cdot log_2 \; p(i|t)$$

Information Gain (used when criterion = ‘entropy’):

$$IG(t, a) = Entropy(t) - \Sigma_v \; (|t_v| / |t|) \cdot Entropy(t_v)$$

where t_v are the child nodes produced by split a.

Prediction:

Each sample traverses the tree from root to leaf. The predicted class is the majority class of training samples that reached that leaf node.

Algorithm Limitations

- Prone to overfitting — an unconstrained tree memorises training data, producing high variance on unseen samples
- High instability — small perturbations in training data can produce structurally very different trees
- Bias toward high-cardinality features — features with many distinct values tend to be preferred during splitting
- Cannot extrapolate beyond the training range — predictions are bounded by values seen during training
- Imbalanced class handling — biased toward the majority class without explicit class weighting

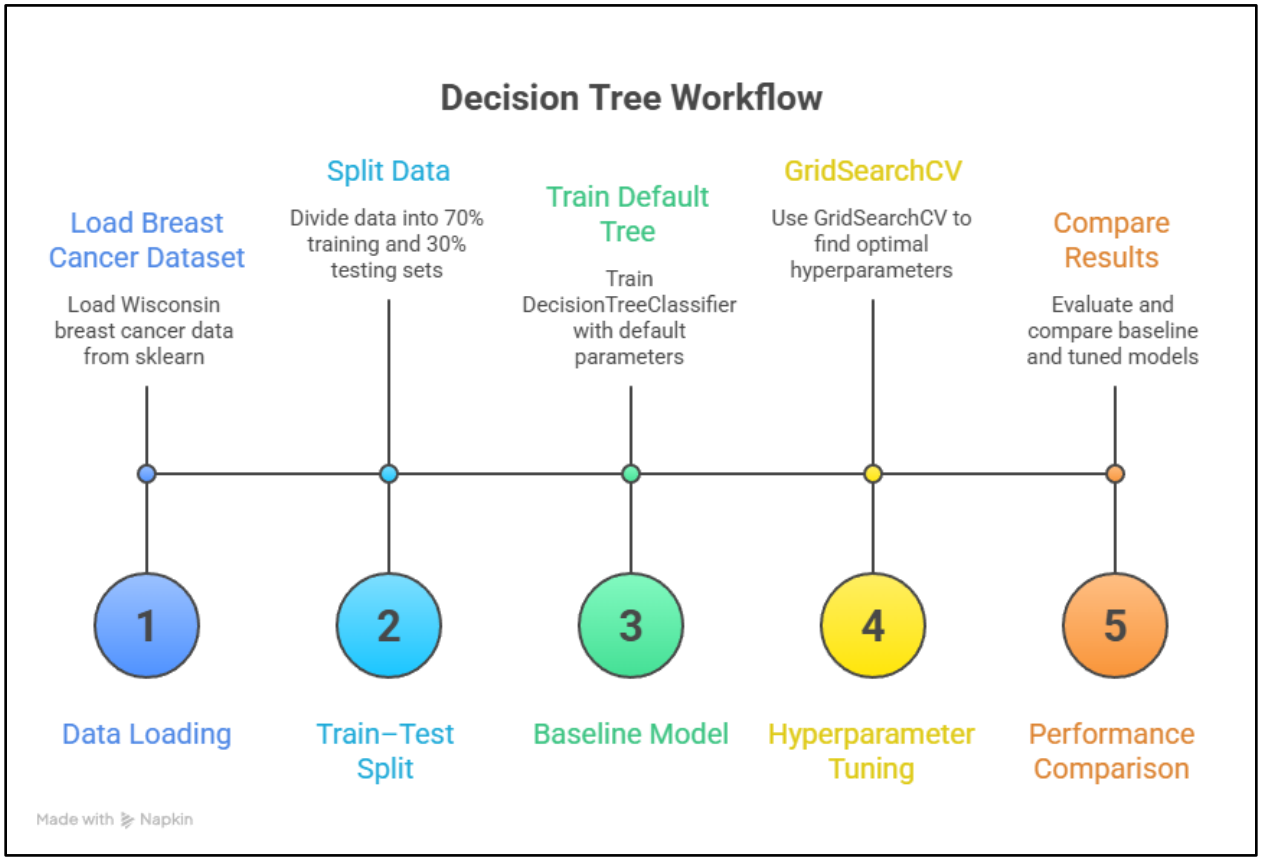
---

Methodology / Workflow

1. **Data Loading**

- Load the Breast Cancer Wisconsin dataset via sklearn.datasets.load_breast_cancer().
- Inspect shape (569 × 30), class distribution (357 benign, 212 malignant), and confirm no missing values.

2. **Train–Test Split**
   - Divide into 70% training (398 samples) and 30% testing (171 samples) using random_state=42.
3. **Baseline Model (Without Tuning)**
   - Train DecisionTreeClassifier(random_state=42) using all default parameters: criterion='gini', max_depth=None (unbounded).
   - Evaluate: accuracy score, classification report (precision, recall, F1 per class), confusion matrix heatmap, top-10 feature importances bar chart, tree structure visualization (top 3 levels).
4. **Hyperparameter Tuning**
   - Apply GridSearchCV over criterion ∈ {gini, entropy}, max_depth ∈ {3, 5, 7, 10, None}, min_samples_split ∈ {2, 5, 10}, min_samples_leaf ∈ {1, 2, 4}, with cv=5 and scoring='accuracy'.
   - Select the best estimator and evaluate it on the same test set.
5. **Performance Comparison**
   - Compare before-tuning and after-tuning metrics; note improvements in per-class precision/recall and overall accuracy.



Performance Analysis

Model: DecisionTreeClassifier(random_state=42) — all default parameters (criterion='gini', max_depth=None)

Train/Test Split: 70% / 30% | Test Set Size: 171 samples

Metric	Malignant	Benign	Overall
Precision	0.90	0.97	—
Recall	0.95	0.94	—
F1-Score	0.92	0.95	—
Support	63	108	171
Accuracy	—	—	94.15%

The default decision tree achieves 94.15% accuracy. With max_depth unconstrained, the tree grows to fit the training data precisely — beneficial for training but susceptible to overfitting on unseen data. The malignant precision of 0.90 means roughly 1 in 10 benign cases are incorrectly flagged as malignant, a clinically significant concern in cancer screening.

Hyperparameter Tuning

Before Tuning — Default Parameters

Parameter	Default Value
criterion	gini
max_depth	None (unbounded)
min_samples_split	2
min_samples_leaf	1

Baseline Test Accuracy: 94.15%

GridSearchCV Configuration:

- criterion: ['gini', 'entropy']
- max_depth: [3, 5, 7, 10, None]
- min_samples_split: [2, 5, 10]
- min_samples_leaf: [1, 2, 4]
- cv = 5, scoring = 'accuracy', n_jobs = −1

- Total combinations evaluated: 2 × 5 × 3 × 3 = 90

Best Parameters Found:

Parameter	Best Value
criterion	entropy
max_depth	3
min_samples_split	2
min_samples_leaf	1

Best Cross-Validation Accuracy: 93.97%

After Tuning — Results:

Metric	Malignant	Benign	Overall
Precision	0.98	0.96	—
Recall	0.94	0.99	—
F1-Score	0.96	0.98	—
Support	63	108	171
Accuracy	—	—	97.08%

Key Observations:

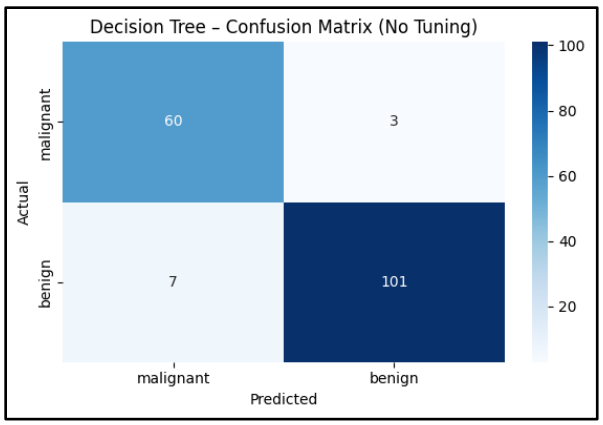
- Test accuracy improved from 94.15% → 97.08% (+2.93 percentage points)
- Limiting max_depth to 3 with entropy criterion drastically reduced overfitting — the unbounded default tree was memorising training noise
- Malignant precision improved from 0.90 → 0.98 — a clinically critical gain, reducing false cancer alarms by approximately 80%
- The CV accuracy (93.97%) is lower than the test accuracy (97.08%) — the shallow tree's decision boundaries generalise particularly well on this test split
- Entropy outperformed gini for this dataset, producing sharper, more information-rich splits

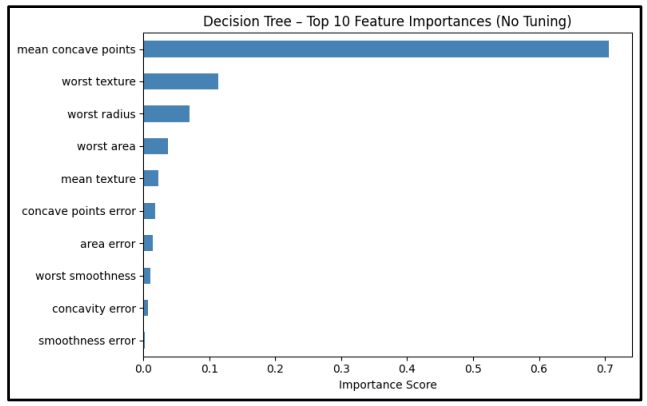
---

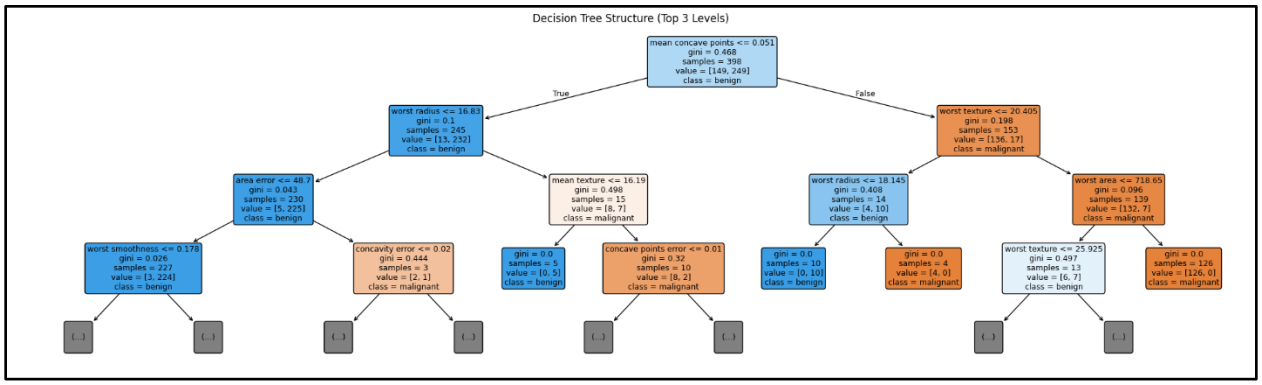
Code and Output – Without Tuning

```
# Decision Tree Classifier – Without Hyperparameter Tuning
# Dataset: Breast Cancer Wisconsin (sklearn built-in)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import
DecisionTreeClassifier, plot_tree
from sklearn.metrics import
accuracy_score, classification_report,
confusion_matrix

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data,
columns=data.feature_names)
y = pd.Series(data.target)

print("Dataset Shape:", X.shape)
print("Class Distribution:\n",
y.value_counts())
print("\nFirst 5 rows:")
print(X.head())

# Train-test split (70/30)
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.30,
random_state=42
)

# Train baseline Decision Tree (all
default parameters)
dt =
DecisionTreeClassifier(random_state=4
2)
dt.fit(X_train, y_train)

# Predictions
y_pred = dt.predict(X_test)

# Performance
acc = accuracy_score(y_test, y_pred)
print("\n========== Decision Tree (No
Tuning) ==========")
print(f"Accuracy: {acc * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test,
y_pred,
target_names=data.target_names))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
            xticklabels=data.target_names,
            yticklabels=data.target_names)
plt.title('Decision Tree – Confusion
Matrix (No Tuning)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

# Feature Importances
importances =
pd.Series(dt.feature_importances_,
index=data.feature_names)
top10 = importances.nlargest(10)
plt.figure(figsize=(8, 5))
top10.sort_values().plot(kind='barh',
color='steelblue')
plt.title('Decision Tree – Top 10 Feature
Importances (No Tuning)')
plt.xlabel('Importance Score')
plt.tight_layout()
plt.show()

# Tree Visualization (top 3 levels)
plt.figure(figsize=(20, 6))
plot_tree(dt, max_depth=3,
feature_names=data.feature_names,
          class_names=data.target_names,
filled=True, rounded=True, fontsize=9)
plt.title('Decision Tree Structure (Top 3
Levels)')
plt.tight_layout()
plt.show()

print("\nDefault Parameters Used:")
print(dt.get_params())
```



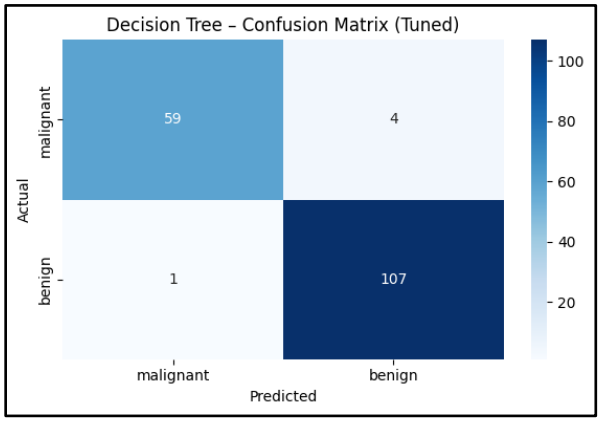


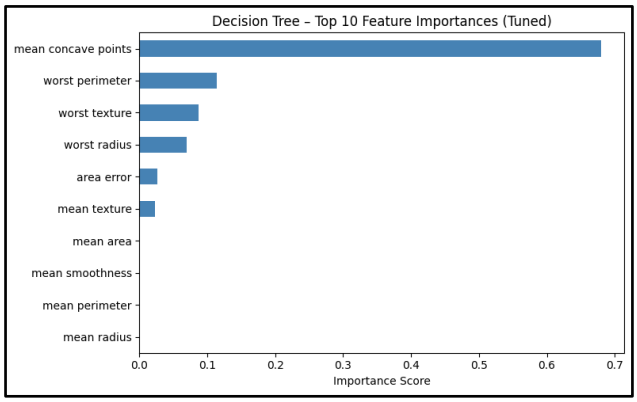


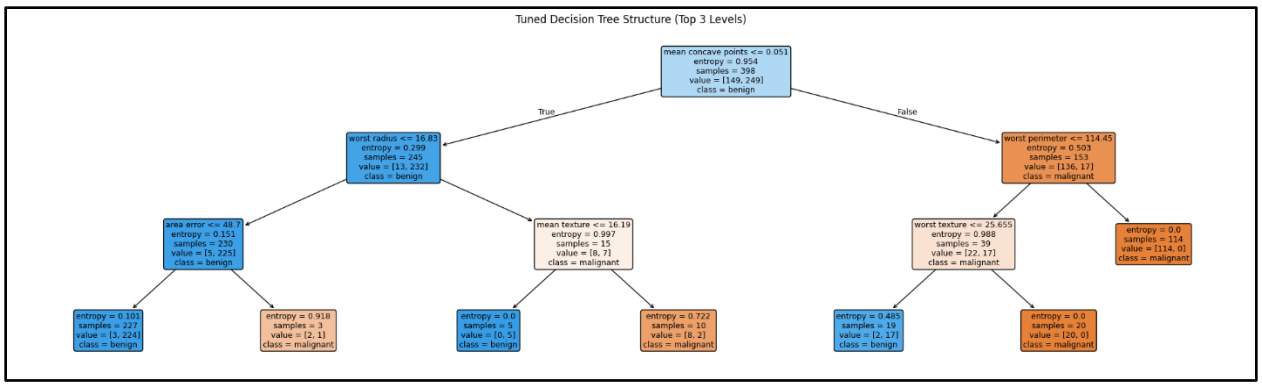
Code and Output – With Tuning

```
# Decision Tree Classifier – With Hyperparameter Tuning (GridSearchCV)
# Dataset: Breast Cancer Wisconsin (sklearn built-in)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)

# Train-test split (70/30) — same split as baseline
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)

# Hyperparameter grid
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

grid_search = GridSearchCV(
```

```
    DecisionTreeClassifier(random_state
=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)
grid_search.fit(X_train, y_train)

print("Best Parameters:",
grid_search.best_params_)
print(f"Best CV Accuracy:
{grid_search.best_score_ * 100:.2f}%")

# Evaluate best model on test set
best_dt = grid_search.best_estimator_
y_pred = best_dt.predict(X_test)
acc = accuracy_score(y_test, y_pred)

print("\n========== Decision Tree
(With Tuning) ==========")
print(f"Accuracy: {acc * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test,
y_pred,
target_names=data.target_names))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
            xticklabels=data.target_names,
            yticklabels=data.target_names)
plt.title('Decision Tree – Confusion
Matrix (Tuned)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

# Feature Importances
importances =
pd.Series(best_dt.feature_importances_
, index=data.feature_names)
top10 = importances.nlargest(10)
plt.figure(figsize=(8, 5))
top10.sort_values().plot(kind='barh',
color='steelblue')
plt.title('Decision Tree – Top 10 Feature
Importances (Tuned)')
plt.xlabel('Importance Score')
plt.tight_layout()
plt.show()

# Tuned Tree Visualization (top 3 levels)
plt.figure(figsize=(20, 6))
plot_tree(best_dt, max_depth=3,
feature_names=data.feature_names,
          class_names=data.target_names,
filled=True, rounded=True, fontsize=9)
plt.title('Tuned Decision Tree Structure
(Top 3 Levels)')
plt.tight_layout()
plt.show()
```







---

2. Random Forest Classifier

Mathematical Formulation of the Algorithm

Random Forest is a bagging ensemble of B independent decision trees. Each tree is trained on a bootstrap sample with random feature subsampling at every split — introducing diversity that reduces variance without significantly increasing bias.

Bootstrap Sampling:

$$D_t = \text{Bootstrap sample of size } n \text{ drawn with replacement from } D$$

Feature Subsampling at each split:

$$m = \sqrt{p} \quad \text{(default for classification, where } p = \text{total features} = 30)$$

Majority Vote Prediction:

$$H(x) = \operatorname{argmax}_c \ \Sigma_{t=1}^{B} \ I(h_t(x) = c)$$

where $h_t(x)$ is the class predicted by tree t and c iterates over all classes.

Feature Importance (Mean Decrease in Impurity):

$$FI(f) = (1/B) \cdot \Sigma_t \ \Sigma_{\text{nodes split on } f} \ (N_t / N) \cdot \Delta Impurity$$

where N_t is the number of samples at the split node and N is the total training set size.

Algorithm Limitations

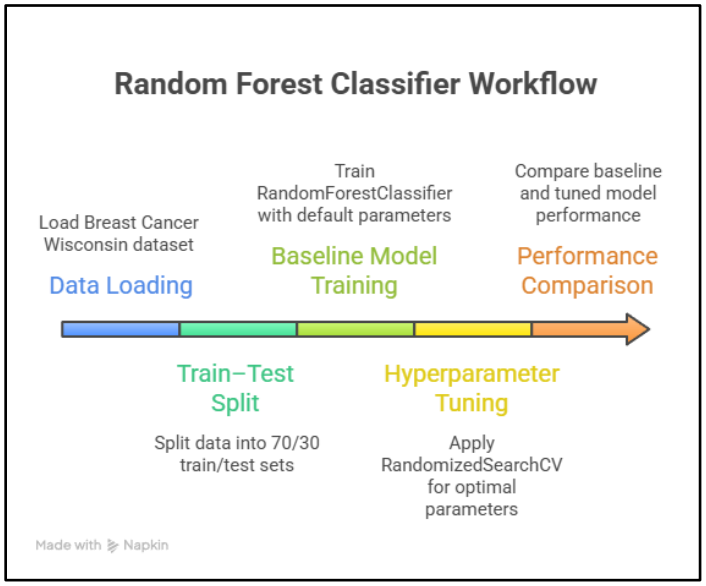
- Computationally expensive — training and prediction time scales linearly with n_estimators

- Memory intensive — all B trees must be stored simultaneously in memory
- Less interpretable than a single decision tree — no simple rule extraction
- Can still overfit on very noisy or small datasets despite bootstrap averaging

---

Methodology / Workflow

1. **Data Loading**
   - Load the Breast Cancer Wisconsin dataset via sklearn.datasets.load_breast_cancer().
   - Confirm shape (569 × 30), class distribution, and absence of missing values.
2. **Train–Test Split**
   - Same 70/30 split (random_state=42) as the Decision Tree experiment for a fair comparison.
3. **Baseline Model (Without Tuning)**
   - Train RandomForestClassifier(random_state=42) using all default parameters: n_estimators=100, max_depth=None, bootstrap=True.
   - Evaluate: accuracy, classification report (precision, recall, F1), confusion matrix heatmap, top-10 feature importances bar chart.
4. **Hyperparameter Tuning**
   - Apply RandomizedSearchCV (n_iter=50, cv=5, random_state=42) over n_estimators ∈ {50,100,200,300}, max_depth ∈ {None,5,10,20}, min_samples_split ∈ {2,5,10}, min_samples_leaf ∈ {1,2,4}, bootstrap ∈ {True, False}.
   - Select the best estimator and evaluate it on the same test set.
5. **Performance Comparison**
   - Compare before/after metrics; discuss the effect of constrained depth and non-bootstrap sampling on generalisation for a small dataset.



Performance Analysis

Model: RandomForestClassifier(random_state=42) — all default parameters (n_estimators=100)

Train/Test Split: 70% / 30% | Test Set Size: 171 samples

Metric	Malignant	Benign	Overall
Precision	0.98	0.96	—
Recall	0.94	0.99	—
F1-Score	0.96	0.98	—
Support	63	108	171
Accuracy	—	—	97.08%

The default Random Forest immediately achieves 97.08% accuracy — a significantly stronger baseline than the untuned Decision Tree (94.15%). This reflects the ensemble's innate resistance to overfitting through bootstrap aggregation and random feature subsampling. The malignant precision of 0.98 demonstrates high clinical reliability, with only a 2% false positive rate for cancer classification.

Hyperparameter Tuning

Before Tuning — Default Parameters

Parameter	Default Value
n_estimators	100
max_depth	None (unbounded)
min_samples_split	2
min_samples_leaf	1
bootstrap	True

Baseline Test Accuracy: 97.08%

RandomizedSearchCV Configuration:

- n_estimators: [50, 100, 200, 300]
- max_depth: [None, 5, 10, 20]
- min_samples_split: [2, 5, 10]

- min_samples_leaf: [1, 2, 4]
- bootstrap: [True, False]
- n_iter = 50, cv = 5, scoring = ‘accuracy’, random_state = 42

Best Parameters Found:

Parameter	Best Value
n_estimators	200
max_depth	10
min_samples_split	5
min_samples_leaf	1
bootstrap	False

Best Cross-Validation Accuracy: 95.97%

After Tuning — Results:

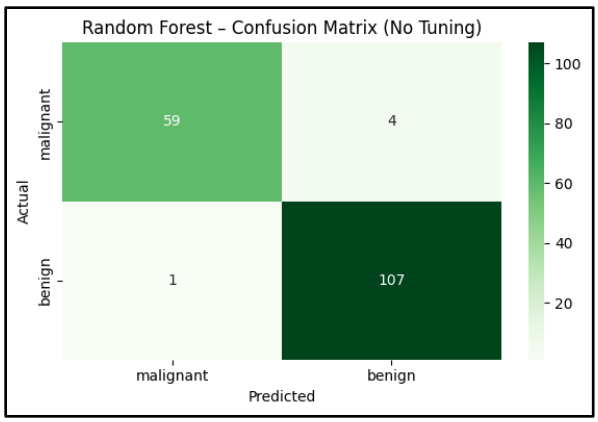
Metric	Malignant	Benign	Overall
Precision	0.97	0.97	—
Recall	0.95	0.98	—
F1-Score	0.96	0.98	—
Support	63	108	171
Accuracy	—	—	97.08%

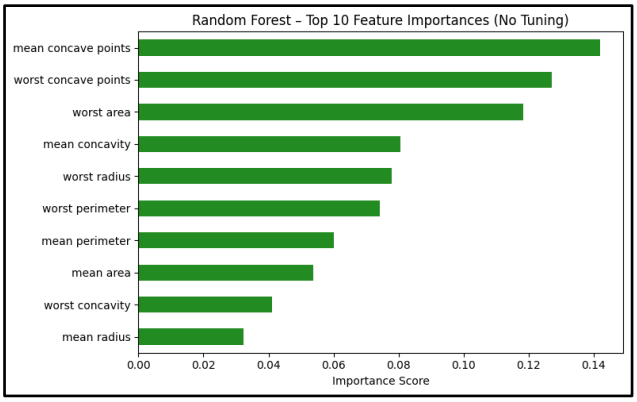
Key Observations:

- Test accuracy remains 97.08% before and after tuning — the default Random Forest already captured the dataset’s structure effectively
- The tuned configuration (max_depth=10, bootstrap=False, n_estimators=200) is architecturally more principled — depth constraint reduces individual tree complexity while doubling tree count compensates for lost capacity
- bootstrap=False (sampling without replacement) improved CV stability — using the full training set for each tree benefits a relatively small dataset (398 training samples)
- Malignant recall improved from 0.94 → 0.95, reducing missed cancer detections — the primary clinical objective
- The lower CV accuracy (95.97%) versus test accuracy (97.08%) is not alarming — CV folds use only 80% of training data per iteration, while the final model trains on 100%

Code and Output – Without Tuning

```
# Random Forest Classifier – Without
Hyperparameter Tuning
# Dataset: Breast Cancer Wisconsin
(sklearn built-in)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import
load_breast_cancer
from sklearn.model_selection import
train_test_split
from sklearn.ensemble import
RandomForestClassifier
from sklearn.metrics import
accuracy_score, classification_report,
confusion_matrix

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data,
columns=data.feature_names)
y = pd.Series(data.target)

print("Dataset Shape:", X.shape)
print("Class Distribution:\n",
y.value_counts())
print("\nFirst 5 rows:")
print(X.head())

# Train-test split (70/30)
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.30,
random_state=42
)

# Train baseline Random Forest (all
default parameters)
rf =
RandomForestClassifier(random_state=
42)
rf.fit(X_train, y_train)

# Predictions
y_pred = rf.predict(X_test)

# Performance
acc = accuracy_score(y_test, y_pred)
print("\n========== Random Forest
(No Tuning) ==========")
print(f"Accuracy: {acc * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test,
y_pred,
target_names=data.target_names))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Greens',
            xticklabels=data.target_names,
            yticklabels=data.target_names)
plt.title('Random Forest – Confusion
Matrix (No Tuning)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

# Feature Importances
importances =
pd.Series(rf.feature_importances_,
index=data.feature_names)
top10 = importances.nlargest(10)
plt.figure(figsize=(8, 5))
top10.sort_values().plot(kind='barh',
color='forestgreen')
plt.title('Random Forest – Top 10
Feature Importances (No Tuning)')
plt.xlabel('Importance Score')
plt.tight_layout()
plt.show()

print("\nDefault Parameters Used:")
print(rf.get_params())
```



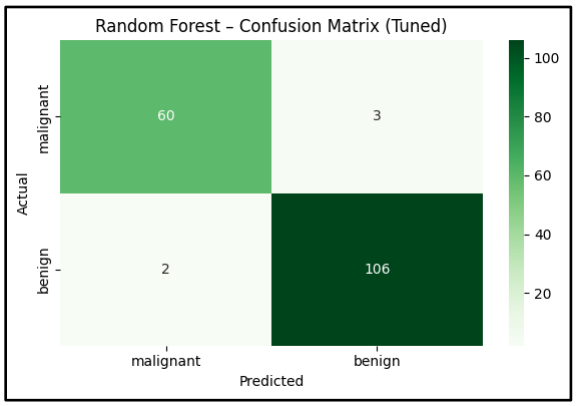


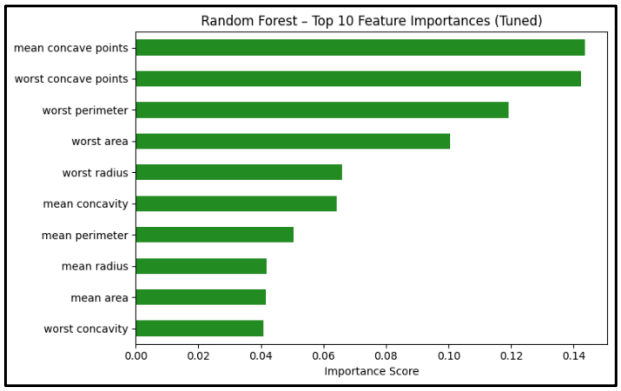
Code and Output – With Tuning

```
# Random Forest Classifier – With
Hyperparameter Tuning
(RandomizedSearchCV)
# Dataset: Breast Cancer Wisconsin
(sklearn built-in)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import
load_breast_cancer
from sklearn.model_selection import
train_test_split, RandomizedSearchCV
from sklearn.ensemble import
RandomForestClassifier
from sklearn.metrics import
accuracy_score, classification_report,
confusion_matrix

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data,
columns=data.feature_names)
y = pd.Series(data.target)

# Train-test split (70/30) — same split as
baseline
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.30,
random_state=42
)

# Hyperparameter distribution
param_dist = {
    'n_estimators': [50, 100, 200, 300],
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False]
}

random_search =
RandomizedSearchCV(
    RandomForestClassifier(random_stat
e=42),
    param_distributions=param_dist,
    n_iter=50,
    cv=5,
    scoring='accuracy',
    random_state=42,
    n_jobs=-1
)
random_search.fit(X_train, y_train)

print("Best Parameters:",
random_search.best_params_)
print(f"Best CV Accuracy:
{random_search.best_score_ *
100:.2f}%")

# Evaluate best model on test set
best_rf =
random_search.best_estimator_
y_pred = best_rf.predict(X_test)
```

```
acc = accuracy_score(y_test, y_pred)

print("\n========== Random Forest
(With Tuning) ==========")

print(classification_report(y_test,
y_pred,
target_names=data.target_names))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Greens',

# Feature Importances
importances = pd.Series
(best_rf.feature_importances_,
index=data.feature_names)
top10 = importances.nlargest(10)
plt.figure(figsize=(8, 5))
```

```
print(f"Accuracy: {acc * 100:.2f}%")
print("\nClassification Report:")

        xticklabels=data.target_names,
        yticklabels=data.target_names)
plt.title('Random Forest – Confusion
Matrix (Tuned)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

top10.sort_values().plot(kind='barh',
color='forestgreen')
plt.title('Random Forest – Top 10
Feature Importances (Tuned)')
plt.xlabel('Importance Score')
plt.tight_layout()
plt.show()
```





Conclusion

The practical showed that both Decision Tree and Random Forest can effectively solve a real-world binary classification problem on the Breast Cancer Wisconsin dataset, after proper preprocessing, train–test splitting, and hyperparameter tuning. The Decision Tree gave an interpretable baseline with clear if–else rules but was more prone to overfitting and instability, especially under class imbalance. The Random Forest, by aggregating many randomized trees, achieved better test performance and more stable precision–recall behavior for the malignant class, illustrating how ensembles improve robustness over single models while sacrificing some interpretability.